Tutorial V.II: Recap with Randomness

Programming: Everyday Decision-Making Algorithms

LECTURER
Dr. Nils Roemer, External Lecturer

AFFILIATION
Planningio GmbH

Introduction

From now on, it is rather difficult to give you tasks that you cannot solve within minutes by yourself with the help of AI. Therefore, today's tutorial will be more open-ended than usual. All tasks are designed to be solved together with AI, but you are nonetheless the human in the loop. Thus, your responsibilities are to control the AI and to learn how to use it by doing. Instructions and explanations will only be given sparsely, as you are encouraged to use AI to ask for help and explanations to explore the tasks on your own.

Let's explore how randomness affects our daily decision-making. As always, we will use Python and simulate various scenarios where randomness plays a crucial role.

Section 1 - Optimal Stopping with Random Candidates

Optimal stopping is a classic problem in decision theory where you want to select the best candidate from a list of candidates. As seen in the lecture, the 37% rule is a simple strategy that can be used to solve this problem.

Exercise 1.1 - The 37% Rule

Let's implement the 37% rule from Optimal Stopping by using random candidates. Your task is to simulate the process of selecting the best candidate out of a list of random candidates. Remember, the 37% rule says that you should reject the first 37% of candidates and then select the next candidate who is better than all previous candidates.

First, let's import the necessary library:

```
import random
```

In total, you have 32 candidates. Screen the candidates one by one and store the index of the candidate you select in `best_candidate_index`.

How to work with AI coding assistants and agents

You can use any current AI coding assistant or agent, for example GitHub Copilot (free for students, see Tutorial V.I), Cursor or Claude Code.

1. Select the task or the code and ask the assistant in its chat to explain the task, the code or a concept.
2. Let it suggest code directly in your notebook and read every suggestion before you accept it.
3. Agents can work on whole files: describe the goal, review their changes and run the tests yourself.
4. If a test fails, ask the assistant why, and make sure you understand the fix.

```
random.seed(13) # Sets the random seed for reproducibility
candidates = [random.random() for _ in range(32)]

# YOUR CODE BELOW
```

```
# Test your solution with the following code
assert best_candidate_index == 19, "The 37% rule should select candidate 19"
print(r"All tests passed, you have successfully implemented the 37% rule!")
print(f"The very best candidate was number {candidates.index(max(candidates))}:
the rule finds the best only about 37% of the time.")
```

Section 2 – Explore vs Exploit

Remember the explore vs. exploit dilemma? Should you try a new restaurant (explore) or return to your favorite (exploit)? Let's **simulate this decision-making process!**

Exercise 2.1 – Visiting Restaurants

Imagine you're new to a city with 10 restaurants. Each restaurant has a "true" quality rating (unknown to you) between 0 and 5 stars. Each time you visit a restaurant, you get to experience its rating. In total, you will visit restaurants 15 times.

Your goal is to create a simulation that computes the average satisfaction over 15 days based on an explore vs. exploit strategy. First, you should try each restaurant once (pure exploration). Then, you should just exploit your favorite restaurant for the remaining visits. Save the average satisfaction in `average_satisfaction` and the index of the best restaurant in `best_restaurant_index`.

```
import random

# Set up the restaurants (true qualities are unknown to the visitor!)
random.seed(59)
qualities = [random.uniform(1, 5) for _ in range(10)]

# YOUR CODE BELOW
```

```
# Test your solution with the following code
assert average_satisfaction > 3.7, "The average satisfaction should be greater
than 3.7"
assert best_restaurant_index == 2, "The best restaurant should be number 3"
print(r"All tests passed, you have successfully implemented the explore vs.
exploit strategy!")
```

Section 3 - Caching

Caching is a technique that can be used to speed up computations by storing the results of expensive computations and reusing them when the same inputs occur again. Let's implement a simple caching mechanism in Python.

Exercise 3.1 - Random Menu Generator

Create a function `generate_daily_menu()` that generates a daily menu by randomly selecting items from different categories. As our "cache" - the daily menu - is rather small, we only want to receive one menu for each day as a list from the function.

```
appetizers = ["Sorted Salad", "Binary Bruschetta", "Array of Antipasti"]
mains = ["Loop Lasagna", "Python Pasta", "Recursive Risotto"]
desserts = ["Binary Brownie", "Cache Cookie", "Stack Sundae"]

# YOUR CODE BELOW
```

```
# Test your solution with the following code
menu = generate_daily_menu()
assert len(menu) == 3, "Menu should contain exactly 3 items"
assert menu[0] in appetizers, "Appetizer should be from the appetizers list"
assert menu[1] in mains, "Main course should be from the mains list"
assert menu[2] in desserts, "Dessert should be from the desserts list"

# Example usage
print(r"All tests passed, you have successfully implemented the menu generator!")
print(f"Today's menu:\nAppetizer: {menu[0]}\nMain: {menu[1]}\nDessert: {menu[2]}")
```

Section 4 – Scheduling

Scheduling is a classic problem in operations research and in daily life. Let's implement a simple scheduling algorithm that tries to find the optimal schedule for a given set of tasks with the earliest due date strategy.

Exercise 4.1 – Scheduling Tasks

Let's create a task scheduler that handles tasks with different due dates. The scheduler should sort tasks by their due date (Earliest Due Date First - EDD strategy) to minimize potential delays.

Your task is to create a function `schedule_tasks(tasks)` that:

- Sorts tasks by due date
- Adds the start and end time of each task in two new columns `start_time` and `end_time`
- Keeps the original columns `name`, `duration` and `due_time`
- Returns the complete schedule as a pandas DataFrame

```
# Pre-generated tasks (don't modify this!)
# Times are represented in hours from start (hour 0)
# For example, due_time: 24 means it's due 24 hours from start
import pandas as pd

tasks = [
    {
        "name": "Project Review",
        "duration": 2,  # hours
        "due_time": 12  # due in 12 hours
    },
    {
        "name": "Team Meeting",
        "duration": 4,
        "due_time": 6
    },
    {
        "name": "Client Presentation",
        "duration": 3,
        "due_time": 7
    },
    {
        "name": "Email Updates",
        "duration": 4,
        "due_time": 11
    },
    {
        "name": "Planning Session",
        "duration": 2,
        "due_time": 9
```

```
    }
]

# YOUR CODE BELOW
```

```
# Test your solution with the following code
schedule_df = schedule_tasks(tasks)
assert schedule_df.shape == (5, 5), "The schedule should have 5 rows and 5
columns"
assert set(schedule_df.columns) == {"name", "duration", "due_time", "start_time",
"end_time"}, "Check the column names"
assert schedule_df.iloc[4]['end_time'] == 15, "The last task should end after 15
hours"
assert schedule_df.iloc[0]['name'] == "Team Meeting", "First task should be 'Team
Meeting'"
assert schedule_df.iloc[4]['name'] == "Project Review", "Fifth task should be
'Project Review'"
print(r"All tests passed, you have successfully implemented the task scheduler!")
```

Section 5 – Randomness

In our daily lives, we often encounter situations where randomness plays a crucial role in decision-making. From choosing a restaurant for dinner to selecting which tasks to tackle first, incorporating some randomness can actually lead to better outcomes than strictly deterministic approaches.

Exercise 5.1 – Monte Carlo Route Planning

Imagine you want to travel during the semester break: starting in Hamburg, you want to visit the other 5 cities once and then return to Hamburg. Instead of checking all possible routes, use a **Monte Carlo simulation** as in the lecture:

1. Create 1,000 random routes (for example with `random.shuffle()` on the list of cities to visit)
2. Compute the cost of each route, including the flights from and back to Hamburg
3. Estimate the **expected cost of a random route** as the average of all simulated costs and save it in `average_cost`
4. Save the cost of the cheapest route you found in `min_cost`

> Tip
>
> The more random routes you simulate, the closer `average_cost` gets to the true average, and the more likely it is that you also find the cheapest route.

```
# City names and cost matrix
import pandas as pd
import random

cities = ["New York", "London", "Tokyo", "Sydney", "Barcelona", "Hamburg"]

cost_matrix = [
    [0, 700, 1500, 2000, 800, 900],  # New York
    [700, 0, 1200, 1800, 300, 700],  # London
    [1500, 1200, 0, 1000, 1100, 1400], # Tokyo
    [2000, 1800, 1000, 0, 1700, 1500], # Sydney
    [800, 300, 1100, 1700, 0, 600],  # Barcelona
    [900, 700, 1400, 1500, 600, 0]   # Hamburg
]

# Create a DataFrame
cost_df = pd.DataFrame(cost_matrix, index=cities, columns=cities)

# Display the DataFrame
print(cost_df)

# YOUR CODE BELOW
```

```
           New York  London  Tokyo  Sydney  Barcelona  Hamburg
New York          0     700   1500    2000        800      900
London          700       0   1200    1800        300      700
Tokyo          1500    1200      0    1000       1100     1400
Sydney         2000    1800   1000       0       1700     1500
Barcelona       800     300   1100    1700          0      600
Hamburg         900     700   1400    1500        600        0
```

```
# Test your solution with the following code
assert abs(average_cost - 6880) < 150, "The average cost of a random route should
be close to 6880"
assert min_cost == 5500, "With 1,000 random routes, you should find the cheapest
route (5500)"
print(r"All tests passed, you have successfully simulated the route planner!")
print(f"A random route costs about {average_cost:.0f} on average, the best one
{min_cost}.")
```

Conclusion

We've explored how randomness affects decision-making through:

- Explore vs. Exploit trade-offs
- Optimal stopping with random elements
- Caching a random subset of data

- Scheduling tasks with due dates
- Monte Carlo simulation of a travel problem

Remember that randomness isn't always bad - sometimes it's the best strategy we have!

In the next tutorial, we'll dive deeper into some free-form tasks!